poly: Checked State Machines and a Governed Runtime for Code Written by Agents

Jean-Jacques Dubray
Cognitive Fab
jdubray@gmail.com

Draft, 7 October 2026

Abstract

Coding agents write correct code for local tasks and fail on stateful code, where defects live in sequences of steps rather than in lines. Formal verification catches such defects, but agents rarely produce verifiable specifications, and the runtimes and policy engines around AI applications check traces after the fact, not programs before they run. We present poly, a language and runtime in which the program an agent writes is itself the model that is checked. A poly program is a set of finite state machines with declared effects, properties and policies. The checker explores every reachable state over finite check domains before the program runs and returns witnesses and typed fixes. The runtime executes only certified programs, passes every effect through a fail-closed gate driven by signed policies, and journals every step so that each run replays with no outside call. A model's answer enters a program as a finite typed value whose every value the checker explores. In a pre-registered experiment of 200 runs on ten stateful tasks, a coding agent with no poly in its training data delivered an admitted program in all 100 poly runs. Escaped defects could not be compared, because the TypeScript baselines had none; every poly defect traced to one language default, which a follow-up experiment changes.

1 Introduction

Coding agents now write a large share of new software. They do well on self-contained tasks and poorly where state, composition and time are involved. A retry that charges a customer twice, a message sent after an opt-out, a cancel that arrives during a payment: each line of such code looks correct, so review misses the defect. The defect lives in the paths between the lines.

Public benchmarks show the pattern. On SlopCodeBench, no agent solved a problem end to end across its iterative checkpoints [49]. On SWE-EVO, long-horizon software evolution scores fall far below single-issue benchmarks [62]. When agents are asked for formal artefacts instead of code, the gap moves rather than closes. Models reach 99% syntactic correctness and 7% verification on multi-function Dafny programs [71]. Across 30 models, 8.6% of generated TLA+ specifications are semantically correct [5]. On SysMoBench, models write TLA+ models of real systems whose syntax passes while conformance to the systems' traces averages about 46% [13].

AI applications add a second difficulty. Some of their steps are model decisions, and some of their effects cannot be undone: a payment, a message to a customer, a change in another system. Durable runtimes such as Temporal and DBOS replay workflows but impose determinism by convention [17, 61]. Agent governance systems such as AgentSpec, Dogwood and CaMeL enforce rules on tool calls at run time [7, 19, 65]. None of them checks the program against its policies before it runs, and none ties the policy to the program's own state.

poly starts from three observations. First, an agent can write formal properties, exhaustive domains and policies, which human authors avoid; the language can therefore ask more of its author. Second, the defects that matter in stateful code are reachable within small finite domains, so exhaustive exploration over declared domains is both feasible and useful. Third, if the checked text is also the executed text, there is no translation from a specification to code for the agent to get wrong.

A poly program is a set of state machines. One source is three things at once: the model the checker explores, the policy every effect must satisfy, and the code the runtime executes. The checker explores every reachable state and returns a witness for each violation. The runtime loads only what the checker admitted, gates every effect, and writes a hash-chained journal from which every state and every gate decision replays.

Contributions.

- A language for agent authors in which the program is the checked model: finite check domains with a strict or disclosed mode, declared effects with classes and labels, five kinds of property, and model decisions as finite typed values (Section 2).
- A checker that folds several engines into one verdict out of five, never reports a bounded search as clean, and returns witnesses that export as host-language tests and typed fixes guarded by node digests (Section 3).
- A runtime that executes only certified programs, validated per program against an independent reference interpreter, with a fail-closed effect gate driven by signed policies and a journal that replays with no outside call (Section 4).
- A pre-registered experiment of 200 runs comparing TypeScript, TypeScript with after-the-fact verification, and two poly surfaces, with its full analysis and the one language default it exposed (Sections 5 and 6).

2 The language

2.1 Machines and finite domains

A machine declares its state, the actions it accepts and a handler per action. A handler either accepts the action and writes the next state, or rejects it with a named reason. Rejection is a value, never an exception, and a rejected step leaves the state unchanged. Figure 1 shows a turnstile.

Three rules make the state space finite and the steps atomic. Every field and parameter has a finite check domain, such as `coins : 0..3` or `amount: int check {10, 20, 500}`. Every `require` comes before any write, so a handler cannot half-apply a step. The frame is derived: a handler writes only the fields it names. Machines lower to the strict profile of SAM v2 [15, 21], in which actions propose, a model accepts or rejects, and the next state is computed from the accepted proposal.

2.2 Strict and disclosed domains

A check domain may be narrower than the runtime type: `amount: int check {10, 20}` checks two amounts. The domain has a mode. In *strict* mode an input outside the domain is rejected at dispatch with the reason `outside-check-domain`, so the checked domain is the running domain. In *disclosed* mode the input is accepted, journaled as drift and listed in the certificate. In the

```
module hello
type Mode = Locked | Unlocked

machine Gate {
  state {
    mode : Mode = Locked
    coins : 0..3 = 0
  }
  action Coin()
  action Push()

  on Coin {
    require coins < 3 else reject "coin-box-full"
    coins' = coins + 1
    mode' = Unlocked
  }
  on Push {
    require mode is Unlocked else reject "locked"
    mode' = Locked
  }
  invariant locked-when-empty: coins == 0 implies mode is Locked
}
```

Figure 1: A turnstile. Each handler states its preconditions before any write. A primed name is the field's value after the step. `Push` does not write `coins`, so the compiler derives that it is unchanged.

version measured in Section 6, the default is strict for every parameter and field whose value can flow into the payload of an irreversible or compensable effect, and disclosed elsewhere. An explicit `disclosed` on a value that reaches an irreversible effect is a finding that only a person can accept.

2.3 Effects

A machine changes the world only by emitting effects. An effect declares a kind, a class (reversible, compensable or irreversible), labels from a closed vocabulary, the targets its handler may call, and the actions dispatched back when it succeeds, fails or exhausts its retries:

```
effect charge(amount: int) {
  kind "payment.charge"
  labels { egress:external }
  targets { egress: "https://api.psp.example/v1/charges" }
  class irreversible
  ok -> PaymentOk()
  failed -> PaymentFailed(code: "declined")
  exhausted -> PaymentFailed(code: "exhausted")
}
```

The host handler receives only the capabilities its labels and targets grant, here an `egress.post` bound to one URL prefix, in the object-capability style [42]. A lint pass refuses handlers that reach for ambient authority such as `fetch`, the file system or the process environment.

2.4 Properties

poly has five kinds of property. An *invariant* holds in every reachable state. A *step property* holds over every accepted step, with `pre` and `post`. A *reject property* holds over every step and its verdict, so a rejection for the wrong reason is a finding. A *path property* holds over every sequence of effects, for example `count("payment.charge") <= 1`. A *liveness* property holds over infinite runs under stated fairness. Deadlock freedom is checked for every machine without being written.

2.5 Model decisions as typed values

A model call is declared like an effect, and its answer must be finite: an enum, an integer range, a three-valued truth, or a record of these. A free-text answer is refused.

```
enum Branch { Refund, Reschedule, Status, Complaint, Other }

decision route(text: string) -> Branch {
  provider "router"
  timeout 800ms
  ok(r) -> Routed(branch: r.answer, conf: r.conf, margin: r.margin)
  failed -> RouteFailed()
}
```

A timeout and a failure completion are required. The confidence and margin may only be compared with constants, so the checker splits each into the intervals the program can tell apart and explores one value per interval. A path property such as "at most one refund" is then proved against every answer the model could give, including a model that answers `Refund` with full confidence to every message. Each confidence threshold reads a per-decision calibration map fitted on adjudicated outcomes. This is a decision-level form of the constraint that type-constrained decoding applies to tokens [47].

2.6 Policies

Policies live in their own files, because different people read them:

```
policy orders-safety {
  scope "orders"
  obligation orders-irreversible
      "No irreversible effect on a cancelled order." rule {
    when effect.class == "irreversible"
         && machine("orders.Order").status == Cancelled
    require false else deny
  }
}
```

Policy expressions are three-valued, because an input may be missing; an unknown never satisfies a requirement. A policy reads the program's state as of the step that emitted the effect. That is the link between governance and the program's meaning, and it is why every gate decision replays.

2.7 Two surfaces

A machine can be written in a standalone syntax or inside TypeScript through an embedded surface. Both produce the same syntax tree and pass the same checks. The experiment of Section 6 selected the embedded surface as the primary one.

3 The checker

3.1 The pipeline

`poly check` lowers each module to the artefacts the engines of the Poly stack read: a contract, a SAM v2 module, invariant files, an effect manifest, path properties and a compiled policy. It then runs its stages in a fixed order: parse, type and domain checks; strict validation of the lowered module; translation validation against the reference semantics; a breadth-first model check of invariants, step and reject properties; deadlock freedom and liveness under stated fairness; path properties over effect sequences; the composition of attached child machines; configurations of communicating machines, with messages in any order within declared bounds; policy admission; obligation consistency, which reports the smallest set of conflicting rules when an obligation cannot be met; a capability proof over effect labels and agent tools; and, when given, a version gate and a replay of recorded traces.

3.2 Five verdicts

A check ends in exactly one verdict. CLEAN: every applicable check ran to completion and found nothing. FINDINGS: at least one property fails, each with a witness. REFUSED: the program cannot be admitted, for example because a policy is malformed. UNDERPOWERED: the checks ran, but the evidence is too thin to support a verdict. INCONCLUSIVE: a search hit its bound, or a property was never exercised. The module verdict is the worst stage verdict, in the order REFUSED, FINDINGS, INCONCLUSIVE, UNDERPOWERED, CLEAN. A stage that did not run does not lower the verdict; it is listed in the certificate with the reason. A bounded search is never reported clean, and every CLEAN is printed with the domains explored and the assumptions it relies on.

3.3 Witnesses and typed fixes

Every finding carries a witness: the shortest path to the violation. For the turnstile of Figure 1, the checker reports a deadlock no one wrote a property for:

```
PL3201 DEADLOCK
witness: Coin, Coin, Coin, Push
state: mode = Locked, coins = 3
Coin is rejected (coin-box-full), Push is rejected (locked)
```

A witness exports as a trace of {`pre, action, data, post`} windows, and as a test in the host language that fails on the defective program and passes on the fixed one. A coding agent does not need to interpret the finding: it runs the test, fixes the program and checks again. Many diagnostics also carry typed fixes, addressed to a node of the syntax tree and guarded by a digest of that node, so a fix whose node an earlier edit changed is refused rather than misapplied. Diagnostics have stable codes and JSON output, and every command is served three ways from one definition: a command line, an MCP server with published schemas, and a language server. Two commands are never served to agents: accepting a finding and approving an escalation. Both require a person's signature.

The loop is the one counterexample-guided synthesis uses, with the agent as the synthesiser [29, 63]. Recent work finds that concrete counterexamples guide models better than scores [52].

4 The runtime

4.1 Admission and translation validation

`poly admit` writes a certificate: the digests of the source, of every lowered artefact and of every check result, signed. The runtime refuses to load an artefact whose digest, signature or toolchain range does not match. The checked program is therefore the running program.

The lowering is not trusted. Every construct is validated per program against `poly-ref`, a reference interpreter of the written semantics that shares no code with the compiler [53]. A lowered module that disagrees with `poly-ref` is a toolchain fault, reported to the toolchain and never to the author. Liveness and deadlock, which no engine artefact covers, are checked by `poly-ref` itself. As a regression measure, the verdicts of a fixed set of 159 programs are recompiled and compared at every milestone; each comparison reported in the project record since the set reached 159 files found them identical.

4.2 One step

A host program creates instances and dispatches actions. `dispatch` runs one step atomically: the new state, the journal entry, the outgoing effects and the timers commit together. It is idempotent on a caller-supplied action identifier, so a request delivered twice is one step. Effects run at least once with a stable idempotency key and are emitted exactly once; an effect that exhausts its retries dispatches its declared failure action.

4.3 The gate

Before it runs, every effect crosses a gate that evaluates it against the policies and against the state of the step that emitted it, never a later one. The outcome is one of three. *Allow*: the handler runs. *Deny*: the handler never runs, and the effect's failure action is dispatched with the reason, so the machine handles a denial like any other failure. *Escalate*: the effect waits for a person, whose approval must be signed with a key registered to a person; an agent's key cannot sign it, and without an approval in time the effect is denied. If the gate is unreachable the effect waits. The runtime never fails open.

Policies are signed. A policy is admitted only with a signature from a keyring whose head, policy sequence numbers and production flag are anchored in a per-user record outside the project, so an agent working in the project cannot sign its own policy, roll a policy back, or move a signature across projects.

On a laptop (Intel Core Ultra 7 155H, Node 24, in-memory SQLite, one worker), the gate's proposal step has a median latency of 0.67 ms and a 99th percentile of 16.0 ms over 146 proposals; the full path from proposal to settled effect has a median of 5.2 ms and a 99th percentile of 21.7 ms. With 64 instances in one trust scope, the runtime settles 128 effects at 111.6 per second.

4.4 Journal and replay

Every step and every gate decision is written as an event. Events are hash-chained per run, each run's head is signed, and a periodic anchor signs a root over all runs, so no run can be dropped from an audit period unnoticed. Only digests of payloads enter the chain; bodies live in a separate store under the operator's retention policy. `poly journal replay` reproduces every state, every gate decision and every model decision from the journal alone: model decisions replay from their recorded answers, and no model is called. Journals export as traces, and `poly check --against`

reports any divergence between what ran and what the program says. This is event sourcing [23] with a checker on both ends.

4.5 Four surfaces for AI applications

Four surfaces extend the same pattern to AI applications. In each, the unbounded part (a provider, a vault, an index, a model) stays in a host adapter, its result enters the program as a finite typed value, the checker explores every value, and the gate enforces the rest.

- **Channels.** Email, WhatsApp, SMS and Slack conversations are machines, so a channel's rules, such as the WhatsApp reply window and opt-outs, are checked on every path.
- **Delegated credentials.** OAuth tokens live in a vault and never reach state, the journal, a prompt or a log. An effect's authority is the overlap of the person's grant, the agent's mandate and the policy, and claims older than their stated age are denied.
- **Governed memory.** Message bodies and retrieved text never enter machine state; a machine holds them as `Content`, an opaque reference. Each body carries a taint (a class and its owners), and the gate refuses a send whose recipient lies outside the audience of what the body was built from. Forgetting a subject erases every body tainted with it, and replay lists the affected steps as forgotten rather than divergent.
- **Routing and budgets.** A cheap model answers first and a stronger one when the calibrated confidence is low. Before each call the gate reserves its worst-case token cost; if the provider reports no usage, the full reservation is charged. Every route ends, and spent plus reserved never exceeds a cap.

These surfaces are implemented on the development branch; their release follows the gates now in progress, and Section 6 measures none of them.

5 Experimental setup

5.1 Question and arms

The experiment asks whether a coding agent that writes poly delivers stateful features with fewer escaped defects than one that writes TypeScript, at an acceptable token cost. It was pre-registered: tasks, oracles, analysis script and decision rules were frozen, by digest, before the runs. One agent implemented the same features behind the same host interface in four arms:

1. TypeScript alone;
2. TypeScript, then polygraph checking after the fact, with fixes allowed;
3. poly, standalone syntax;
4. poly, embedded TypeScript surface.

The poly arms received the language manual in their workspace. No poly code exists in any model's training data.

5.2 Tasks, runs and oracles

There were ten stateful tasks (payments with timeouts and partial captures, leases, job retries, order workflows and others). Each task is a base version and three changes delivered in sequence

Measure (mean over tasks of per-task means)	Arm 1	Arm 2	Arm 3	Arm 4
Escaped defects per delivered task (O1)	0.00	0.00	1.04	1.08
Tokens per delivered task	496,146	4,267,380	1,746,467	1,737,013
Runs delivered within budget	50/50	47/50	50/50	50/50
Admitted (delivered and CLEAN)			50/50	50/50

Table 1: The pre-registered measures of the experiment. Arms: 1 TypeScript; 2 TypeScript then polygraph; 3 poly standalone; 4 poly embedded.

to the same run, so a run delivers four versions. Each task and arm was run five times, in blocks of the four arms started together: 50 blocks and 200 runs. Every run used `claude-opus-5-5` through the Claude CLI, with caps of 15,000,000 tokens and 3,600 s per run.

The primary oracle (O1) is a suite of scenario tests written from the briefs by an independent session that saw no reference solution. Secondary oracles are hidden invariants (O2), reference traces (O3) and range-refusal tests (O1-R). An escaped defect is a failing O1 test in a delivered version.

5.3 Decision rules

Win: a poly arm reduces mean escaped defects per delivered task by at least 30% against both arms 1 and 2, with tokens per delivered task at most twice arm 1's, and both Holm-corrected [25] comparisons below 0.05 (exact two-sided Wilcoxon signed-rank tests over the ten tasks [68]). *Stop*: neither poly arm delivers an admitted program (delivered, with verdict CLEAN) in 70% of its runs within budget. Anything between is reported as it is. A *surface rule* picks the embedded surface if its defects are within 10% of the standalone surface's and its tokens are no higher.

6 Results

6.1 Verdict and primary measure

All 200 runs were recorded and scored, and the frozen analysis returns *between*: no win and no stop. Table 1 gives the measures.

Arms 1 and 2 had no failing O1 test in any delivered run. The primary measure therefore had no room to show a reduction, and the win rule, which requires a comparator mean above zero, could not be met. No escaped-defect comparison is significant (all Holm-corrected $p = 1.0$). Post hoc bootstrap intervals over tasks (10,000 resamples) for the differences between poly arms and baselines run from 0.00 to about 3.1. With ten tasks the experiment detects only large differences, so the result is unresolved, not equal.

6.2 Learnability

Both poly arms delivered an admitted program in 50 of 50 runs, so the stop rule did not fire. A coding agent learned a language absent from its training data from a manual in context and checker feedback, and every delivered program passed the full check. This matches recent evidence that agents can acquire an unfamiliar language at inference time from documentation and an execution environment [55], here with a checker as the environment.

6.3 Tokens

Both poly arms used about 3.5 times arm 1's tokens per delivered task, beyond the win rule's bound of twice, and about 0.41 times arm 2's (all four comparisons Holm-corrected $p = 0.0098$). Writing the properties and answering the checker costs tokens; verifying TypeScript after the fact cost more. The two poly surfaces did not differ (0.5%, Holm-corrected $p = 0.92$), and the surface rule selected the embedded surface.

6.4 Where the poly defects came from

Every escaped defect of the poly arms is in two tasks. In T3 (card payments), two of five runs in each poly arm accepted a capture or refund of zero or of a negative amount, and a refund of 3 units, outside the range the brief states: the same 26 O1 tests failed in all four runs. In T7, one run of arm 4 accepted a timeout for a task index outside its range. In each case the parameter had a check domain, but its value did not reach an irreversible or compensable effect, so the default mode was disclosed: the checker explored the domain, and the runtime accepted values outside it. The checked domain was not the running domain. One default explains every poly defect of the experiment.

Pending. Strict by default (experiment SR, pre-registered). Part 1 recompiles the five defective deliveries with only the default changed to strict for every value with a check domain, and reruns O1 and O1-R. Part 2 reruns T3 and T7 in both poly arms, five runs each, and compares runs with an escaped defect against the same cells of the experiment (5 of 20) with a one-sided Fisher exact test; zero of 20 gives $p = 0.024$. Results to be inserted here.

6.5 Secondary observations

Two secondary findings are not escaped defects. In T1, both poly arms emitted a refund that the brief says "needs no answer" as an effect, which then stayed pending; O3 flags 216 traces per run and O1 has no test of it. In T10, every arm of every run follows the third change of the brief where the reference follows the base brief; the brief says both. Arm 2 alone moved long-running checker commands to the background after its tool timeout, and its three runs that missed the budget all stopped at the wall-clock cap.

7 Related work

Model-checked languages. P models distributed systems as communicating state machines and checks them, and is used at scale at AWS [20]. TLA+ and Alloy check specifications [28, 34], and Quint positions an executable specification as the validation point between English and code for LLM-written software [45]. In each, the checked model is separate from the production code. Stateright model-checks the Rust actor code it runs [57], which is the closest analogue to a checked program that is also the running program; it has no agent tooling or governance. poly descends from explicit-state model checking [14, 26] and statecharts [24].

Verified code written by models. Dafny is the most model-friendly verification target [8, 40], with Verus and Lean behind it [72, 73]; success collapses under composition [71]. Lean has been proposed as the independent verification layer for AI-written code [18]. poly trades proof over unbounded domains for exhaustive exploration over declared finite ones, which an agent completes without proof engineering.

Verifier feedback. Lemur, Clover, Baldur and AlphaVerus use verifier or checker output to steer generation and repair [1, 22, 58, 69]. ESBMC-AI feeds bounded model-checker counterexamples to a model and re-verifies each fix [63], and agentic model checking leaves every soundness decision to the checker [59]. poly makes this loop the language's interface: witnesses as tests, typed fixes, and verdicts an agent can act on.

Languages for models and agents. BAML, DSPy and LMQL type or constrain model calls [3, 6, 32]; type-constrained decoding constrains generation itself [47]. Zero and MoonBit design toolchains for agent authors, with structured diagnostics and typed repairs [44, 60]. Turn gates control flow on typed inference with confidence and keeps credentials out of agent memory [33]; Pel constrains agent orchestration by syntax [43]. None checks state machines before execution. Language choice affects model correctness [9, 10, 64], which makes the learnability result of Section 6 a precondition for everything else.

Durable runtimes and replay. Temporal, DBOS, Step Functions and LangGraph journal or checkpoint workflows [2, 17, 35, 61]; Step Functions alone declares a state machine, without invariants. Record and replay of agent traffic is studied at the network level and at cut points [11, 12, 46]. In poly the journal is the replay log, the audit record and the input to the next check.

Agent security and governance. AgentSpec, Invariant Guardrails, Dogwood and PCAS enforce rules on agent traces at run time [7, 27, 51, 65]. CaMeL, FIDES, Progent, ACE and the design-pattern catalogue of Beurer-Kellner et al. separate trusted control from untrusted data and enforce capability or information-flow policies [4, 16, 19, 36, 56]; MemLineage applies lineage to agent memory [50]. Tool-poisoning attacks succeed often enough that a gate outside the model is required [67]. poly's governed memory follows the information-flow line, with taint checked on every path before the run and on every send during it.

Nearest neighbours. CaMeLoT models an agent's plan as a transition system, checks it against temporal policies before any tool runs and returns counterexamples [48]. Agentproof statically checks workflow graphs extracted from agent frameworks [70], Lean4Agent models workflows in Lean [66], and Agent-C enforces temporal constraints during generation [30]. These check a plan or a graph; poly checks the whole program state space of the code that runs, and ties the runtime gate and the replay log to the same source.

Evaluation. Statistical practice for model evaluations [41], cost-aware agent evaluation [31], benchmark rigour [74] and reliability metrics [54] shaped the design of Section 5. Harness studies find that, for a fixed model, the harness mostly moves token cost [37, 39], and VERA builds verifiable environments that restart at any stage and score each stage from evidence [38]; a poly program provides stages, restarts and checks by construction.

8 Limitations

- **The floor.** The baselines had no escaped defects, so the experiment could not measure what poly prevents. Harder tasks are needed, with the smallest detectable gap stated before the runs.

- **Ten tasks, one model.** Results hold for `claude-opus-5-5` on ten tasks; other models and harnesses are untested.
- **Finite domains.** Checks are exhaustive over declared domains only. A property that fails only outside them is not found; the domains are disclosed in every verdict, and strict mode makes them the running domains.
- **Tokens.** poly cost 3.5 times the tokens of plain TypeScript per delivered task in this experiment.
- **Surfaces unmeasured.** Channels, credentials, memory and routing are not covered by the experiment.
- **One runtime.** poly runs in TypeScript on Node; the checker engines and the reference interpreter are by the same author.

9 Conclusion

poly makes the program an agent writes the model that is checked, the policy that governs it and the code that runs. Its checker explores every reachable state over finite domains and answers with witnesses and typed fixes; its runtime executes only certified programs, gates every effect without failing open, and replays every run from its journal with no outside call. A model's answer is a finite typed value the checker has already explored.

In a pre-registered experiment of 200 runs, a coding agent with no poly in its training data delivered an admitted program in every poly run. The baselines had no escaped defects on these tasks, so the experiment did not measure what poly prevents; it did locate every poly defect in one default, the mode of a check domain whose value reaches no irreversible effect. The next steps follow: strict domains by default and its pre-registered test, and a second experiment on tasks hard enough to separate the arms.

References

[1] Pranjal Aggarwal, Bryan Parno, and Sean Welleck. AlphaVerus: Bootstrapping formally verified code generation through self-improving translation and treefinement. arXiv:2412.06176, 2024.

[2] Amazon Web Services. AWS Step Functions and the Amazon States Language. `https://aws.amazon.com/step-functions/`, 2026.

[3] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proc. ACM Program. Lang. (PLDI)*, 7, 2023.

[4] Luca Beurer-Kellner et al. Design patterns for securing LLM agents against prompt injections. arXiv:2506.08837, 2025.

[5] Arslan Bisharat et al. Can LLMs write correct TLA+ specifications? evaluating natural-language-to-TLA+ generation. In *International Conference on Software Technologies (ICSOFT)*. SciTePress, 2026. doi: 10.5220/0015070400004088. arXiv:2606.05792.

[6] BoundaryML. BAML: the programming language for agents. `https://www.boundaryml.com`, 2026.

[7] Marc Brooker, Joseph Tassarotti, and Jean-Baptiste Tristan. Introducing Dogwood: Runtime verification for AI agents. AWS Open Source Blog, 6 August 2026, https://aws.amazon.com/blogs/opensource/introducing-dogwood-runtime-verification-for-ai-agents/, 2026.

[8] Sergiu Bursuc et al. A benchmark for vericoding: Formally verified program synthesis. Dafny Workshop at POPL 2026, 2026. arXiv:2509.22908.

[9] Federico Cassano et al. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3675–3691, 2023.

[10] Federico Cassano et al. Knowledge transfer from high-resource to low-resource programming languages for code LLMs. In *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2024.

[11] Tisha Chawla and Susheem Koul. Chronicle: Cut-point replay for regression testing of LLM agents. arXiv:2609.20625, 2026.

[12] Yunji Chen, Shijin Zhang, Qi Guo, Ling Li, Ruiyang Wu, and Tianshi Chen. Deterministic replay: A survey. *ACM Computing Surveys*, 48(2):1–47, 2015. doi: 10.1145/2790077.

[13] Qian Cheng et al. SysMoBench: Evaluating AI on formally specifying complex real-world systems. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.23130.

[14] Edmund M. Clarke and E. Allen Emerson. Design and synthesis of synchronization skeletons using branching-time temporal logic. In *Logic of Programs*, volume 131 of *LNCS*, pages 52–71, 1982.

[15] Cognitive Fab. SAM v2: the State-Action-Model library (@cognitive-fab/sam-lib). npm package, 2026.

[16] Manuel Costa et al. Securing AI agents with information-flow control. arXiv:2505.23643, 2025.

[17] DBOS, Inc. DBOS: Durable workflows on Postgres. https://www.dbos.dev, 2026.

[18] Leonardo de Moura. When AI writes the world's software, who verifies it? Blog post, 28 February 2026, https://leodemoura.github.io/blog/2026-2-28-when-ai-writes-the-worlds-software-who-verifies-it/, 2026.

[19] Edoardo Debenedetti et al. Defeating prompt injections by design. arXiv:2503.18813, 2025.

[20] Ankush Desai, Vivek Gupta, Ethan Jackson, Shaz Qadeer, Sriram Rajamani, and Damien Zufferey. P: Safe asynchronous event-driven programming. In *Programming Language Design and Implementation (PLDI)*, pages 321–332, 2013.

[21] Jean-Jacques Dubray. Why i no longer use MVC frameworks. InfoQ, https://www.infoq.com/articles/no-more-mvc-frameworks/, 2016.

[22] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023.

[23] Martin Fowler. Event sourcing. `https://martinfowler.com/eaaDev/EventSourcing.html`, 2005.

[24] David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, 1987.

[25] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[26] Gerard J. Holzmann. The model checker SPIN. *IEEE Transactions on Software Engineering*, 23(5):279–295, 1997.

[27] Invariant Labs. Invariant guardrails. `https://invariantlabs.ai`, 2025.

[28] Daniel Jackson. *Software Abstractions: Logic, Language and Analysis*. MIT Press, revised edition, 2012.

[29] Sumit Kumar Jha et al. Neuro symbolic reasoning for planning: Counterexample guided inductive synthesis using large language models and satisfiability solving. arXiv:2309.16436, 2023.

[30] Adharsh Kamath, Sishen Zhang, Calvin Xu, Shubham Ugare, Gagandeep Singh, and Sasa Misailovic. Enforcing temporal constraints for LLM agents. arXiv:2512.23738, 2025.

[31] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. arXiv:2407.01502, 2024.

[32] Omar Khattab et al. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.

[33] Muyukani Kizito. Turn: A language for agentic computation. arXiv:2603.08755, 2026.

[34] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[35] LangChain. LangGraph. `https://github.com/langchain-ai/langgraph`, 2026.

[36] Evan Li, Tushin Mallick, Evan Rose, William Robertson, Alina Oprea, and Cristina Nita-Rotaru. ACE: A security architecture for LLM-integrated app systems. In *Network and Distributed System Security Symposium (NDSS)*, 2026.

[37] Haozhe Liu et al. SoL-Pi: Recursively scaling auto-research loops for efficient agent harness. arXiv:2609.20519, 2026.

[38] Junqi Liu et al. VERA: Scaling verifiable environments for agentic co-evolution. arXiv:2610.05923, 2026.

[39] Yangze Liu and Zhongyi Han. What does a harness buy? tokens, mostly. arXiv:2610.04433, 2026.

[40] Chloe R. Loughridge et al. DafnyBench: A benchmark for formal software verification. *Transactions on Machine Learning Research*, 2025. arXiv:2406.08467.

[41] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. arXiv:2411.00640, 2024.

[42] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.

[43] Behnam Mohammadi. Pel, a programming language for orchestrating AI agents. arXiv:2505.13453, 2025.

[44] MoonBit team. MoonBit: an AI-native programming language toolchain. `https://www.moonbitlang.com`, 2026.

[45] Gabriela Moreira. Reliable software in the LLM era. Quint blog, 17 November 2025, `https://quint.sh/posts/llm_era`, 2025.

[46] Rasheed Mudasiru. Deterministic replay for AI agent systems. arXiv:2607.16200, 2026.

[47] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. Type-constrained code generation with language models. In *Programming Language Design and Implementation (PLDI)*, 2025. doi: 10.1145/3729274.

[48] Elia Nikolaou, Magnus Wiik Eckhoff, Robert Flood, Gudmund Grov, Vasileios Mavroeidis, and David Aspinall. CaMeLoT: CaMeL orchestrated with temporal logic for static verification and liveness. arXiv:2609.18674, 2026.

[49] Gabriel Orlanski et al. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. arXiv:2603.24755, 2026.

[50] Ciyan Ouyang and Rui Hou. MemLineage: Lineage-guided enforcement for LLM agent memory. arXiv:2605.14421, 2026.

[51] Nils Palumbo, Sarthak Choudhary, Jihye Choi, Prasad Chalasani, Mihai Christodorescu, and Somesh Jha. Policy compiler for secure agentic systems. arXiv:2602.16708, 2026.

[52] André G. Pereira, Augusto B. Corrêa, and Jendrik Seipp. Property-guided LLM program synthesis for planning. arXiv:2605.16142, 2026.

[53] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 1384 of *LNCS*, pages 151–166, 1998. doi: 10.1007/BFb0054170.

[54] Stephan Rabanser et al. Towards a science of AI agent reliability. arXiv:2602.16666, 2026.

[55] Chen Shen, Wei Cheng, Jingyue Yang, Huan Zhang, Yuhan Wu, and Wei Hu. Bridging the knowledge void: Inference-time acquisition of unfamiliar programming languages for coding tasks. arXiv:2602.06976, 2026.

[56] Tianneng Shi et al. Progent: Programmable privilege control for LLM agents. arXiv:2504.11703, 2025.

[57] Stateright contributors. Stateright: A model checker for implementing distributed systems. `https://github.com/stateright/stateright`, 2026.

[58] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. Clover: Closed-loop verifiable code generation. arXiv:2310.17807, 2023.

[59] Youcheng Sun, Jiawen Liu, Daniel Kroening, and Jason Xue. Agentic model checking. arXiv:2605.21434, 2026.

[60] Chris Tate. Zero: an experimental systems language for agents. Vercel Labs, May 2026, `https://zerolang.ai`, `https://github.com/vercel-labs/zero`, 2026.

[61] Temporal Technologies. Temporal: Durable execution. `https://temporal.io`, 2026.

[62] Minh V. T. Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. arXiv:2512.18470, 2025.

[63] Norbert Tihanyi et al. A new era in software security: Towards self-healing software via large language models and formal verification. arXiv:2305.14752, 2023.

[64] Lukas Twist et al. A study of LLMs' preferences for libraries and programming languages. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 331–351, 2026.

[65] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. arXiv:2503.18666.

[66] Ruida Wang, Jerry Huang, Pengcheng Wang, Xuanqing Liu, Luyang Kong, and Tong Zhang. Lean4Agent: Formal modeling and verification for agent workflow and trajectory. arXiv:2606.06523, 2026.

[67] Zhiqiang Wang et al. MCPTox: A benchmark for tool poisoning attack on real-world MCP servers. arXiv:2508.14925, 2025.

[68] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

[69] Haoze Wu, Clark Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. In *International Conference on Learning Representations (ICLR)*, 2024.

[70] Melwin Xavier, Vaisakh M A, Melveena Jolly, and Midhun Xavier. Agentproof: Static verification of agent workflow graphs. arXiv:2603.20356, 2026.

[71] Xu Xu, Xin Li, Xingwei Qu, Jie Fu, and Binhang Yuan. Local success does not compose: Benchmarking large language models for compositional formal verification. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.23061.

[72] Chenyuan Yang et al. AutoVerus: Automated proof generation for Rust code. In *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2025. arXiv:2409.13082.

[73] Zhe Ye, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song. VERINA: Benchmarking verifiable code generation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2505.23135.

[74] Yuxuan Zhu et al. Establishing best practices in building rigorous agentic benchmarks. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. arXiv:2507.02825.

A Glossary

Admitted
Delivered with a CLEAN verdict and bound to a signed certificate.

Certificate
The signed record binding a source to its lowered artefacts and check results.

Check domain
The finite set of values a field or parameter is checked over.

Content
An opaque reference to a message body or retrieved text, never read by a machine.

Decision
A model call whose answer is a finite typed value.

Disclosed
A domain mode in which out-of-domain inputs are accepted and journaled as drift.

Effect
A declared action on the outside world, emitted by an accepted step and passed through the gate.

Escaped defect
A failing scenario test (O1) in a delivered version.

Gate
The runtime component that allows, denies or escalates each effect, and never fails open.

Journal
The hash-chained, signed record of every step and gate decision.

Machine
A declared state machine: state, actions, handlers, effects and properties.

poly-ref
The reference interpreter of the written semantics, sharing no code with the compiler.

Strict
A domain mode in which out-of-domain inputs are rejected at dispatch.

Taint
The class and owners carried by content built from governed facts.

Translation validation
Checking each compiled program against the reference semantics, instead of trusting the compiler.

Witness
The shortest replayable sequence of steps that shows a finding.